

Architectural Overview & Component Descriptions

The **Saudi Legal Document Crawler** is engineered as a robust, production-grade data ingestion framework designed to automate the lifecycle of legal document collection, validation, and normalization. The system architecture heavily enforces the **Separation of Concerns (SoC)** principle and incorporates a **Fail-Fast Governance Mechanism** to guarantee complete license compliance and data purity before storage.

1. Governance Layer (`sources_registry.py`)

Serving as the system's **Single Source of Truth**, this centralized configurations registry decouples target portal definitions from the core crawling logic.

- **Mechanics:** It maintains a comprehensive list of approved domains, metadata categories, and explicit licensing parameters.
- **Scalability Impact:** Any future expansion of target sources requires zero modifications to the underlying source code. A new government domain can be registered dynamically as a simple entry profile within this layer.

2. Gate & Traffic Control Layer (`middlewares.py` & `settings.py`)

This defensive boundary intercepts and evaluates network operations *prior* to executing outgoing requests, shielding target servers and enforcing defensive scraping behavior.

- **DomainWhitelistMiddleware :** Implements an early mitigation check by auditing target request URLs directly against the approved registry. Unsanctioned or non-`.sa` domains are dropped instantly, preventing accidental scope creep or unintended crawling.
- **Engine Settings Control Room:** Centralizes the configuration for polite scraping. It uses `AutoThrottle` and calculated concurrency delays to respect the server limits of government portals dynamically.
- **Custom Downloader Middleware:** Dynamically rotates incoming `User-Agent` strings to replicate realistic client sessions and handles transient network drops via an automated backend retry matrix.

3. Extraction Layer (`spiders/`)

Designed purely for targeted data navigation, parsing, and extraction, this layer handles raw web presentation structures.

- **BaseSaudiSpider :** An abstract parent class that embeds shared crawler configurations and inherits global governance validations.

- **Granular Spiders (`boe_spider.py` , `uqn_spider.py` , etc.):** Decoupled, site-specific sub-classes whose *sole responsibility* is navigating the Document Object Model (DOM) of their respective platforms. Isolating each portal into its own spider ensures high fault-tolerance; if a single website alters its HTML structure, only its isolated spider breaks, leaving the rest of the ecosystem fully functional.

4. Data Contract Layer (`items.py`)

- **SaudiLegalDocumentItem :** Establishes a rigid **Data Contract** mapping the exact relational boundary between the extraction spiders and the processing pipelines. It standardizes expected payload keys—such as document IDs, raw dates, titles, and licensing flags—ensuring that downstream processors receive structurally uniform data objects regardless of the source origin.

5. Post-Processing Pipeline Layer (`pipelines.py`)

Operating as an uncoupled sequential processing chain, this layer subjects each emitted `Item` to deep sanitization, enrichment, and binary extraction stages:

- **ValidationPipeline :** Executes type checking, structural validation, and verifies that the item complies with the required metadata fields and specific category enums.
- **LicenseCompliancePipeline :** Performs a second-tier governance audit. If an item stems from an uncertain or ambiguous origin, it flags the record with a `review_flag=True` value to safeguard the primary training set.
- **DateNormalizationPipeline :** Resolves the discrepancy between mixed Hijri and Gregorian calendar strings found across different portals by parsing and converting raw texts into a uniform, ISO-standardized `YYYY-MM-DD` formatting required for LLM training inputs.
- **SldFilesPipeline & DeduplicationEngine :** Coordinates asynchronous streaming downloads of attached binary PDFs. Rather than trusting filename conventions, it computes an on-the-fly **SHA-256 content-hash checksum** of the downloaded file. Existing hashes are filtered out immediately to guarantee zero duplicate bloat and maximize storage efficiency.
- **Target Delivery (`CSVExportPipeline` & `AuditLogPipeline`):** Splitting mechanisms write compliant rows safely to the relational catalog, while quarantined metadata profiles containing strict review flags are rerouted instantly into a separate inspection ledger.

6. Persistence & Storage Layer

The terminal layer organizes all structured assets into clear relational and physical directory taxonomies:

- **metadata.csv** : A clean, structurally consistent relational database index containing highly structured, machine-ready metadata fields.
- **corpus/ Storage Directory**: A strictly normalized, predictable file system layout organized dynamically as: `corpus/{category}/{source}/{doc_id}.pdf` .
- **review_queue.csv** : An isolated quarantine ledger containing flagged items requiring manual human sign-off before their respective files are merged into the clean master corpus.